Onyx — SystemKit reference

Onyx · multi-process OS on Raspberry Pi 4

*The reference of **SystemKit** (`SD:/lib/systemkit.so`, `user/Kits/systemkit`): what it is for, how a program uses it, and every operation it exposes. The operations' part is made from the kit's headers by `tools/docgen/kitdocs.py` — the headers are the source. Overview of all the kits: The Kits.*

Contents

1. What it is
2. Using it
3. Index
4. `systemkit/notify.h`
5. `systemkit/clipboard.h`
6. `systemkit/clipproto.h`
7. `systemkit/trash.h`
8. `systemkit/fileassoc.h`
9. `systemkit/volume.h`
10. `systemkit/wallpaper.h`
11. `systemkit/dockconf.h`
12. `systemkit/preloadini.h`
13. `systemkit/autostart.h`
14. `systemkit/locale.h`
15. `systemkit/session.h`
16. `systemkit/applet_proto.h`

What it is

SystemKit is what a program says to the system and to the other programs: notifications, the clipboard, the trash, the file associations, the volume, the wallpaper, the dock, the programs loaded ahead and started at boot, the Control Panel's applets.

Include	`#include "systemkit/systemkit.h"`
Link	`lib/systemkit.imp.a` (C++) or `lib/systemkit.imp_c.a` (C)
Library	`SD:/lib/systemkit.so` — 119 entries in its table (`user/Kits/systemkit/systemkit.abi`, append-only)
Sources	`user/Kits/systemkit/`

Using it

A notification (a bubble under the menu bar; `notify_action` adds what a click starts):

```
#include "systemkit/systemkit.h"

notify ("Backup", "3 files copied");
notify_action ("Updates", "2 updates available", "control pkgman");
```

The clipboard, shared by every application:

```
#include "systemkit/systemkit.h"

clip_set_text ("copied text");

char buf[1024];
if (clip_get_text (buf, sizeof buf)) { /* paste buf */ }

int w, h;
unsigned *px = clip_get_image (&w, &h);              // a picture, if one was copied
if (px) { /* ... */ delete [] px; }
```

The trash — a file is moved there instead of being deleted:

```
#include "systemkit/systemkit.h"

if (trash_move ("SD:/docs/old.txt")) notify ("Trash", "old.txt moved to the trash");
int n = trash_count ();                              // what the trash holds
```

Opening a file with the right application (the user's file associations):

```
#include "systemkit/systemkit.h"

char app[32];
if (fa_app_for ("SD:/photos/cat.png", app, sizeof app)) { /* "imageview" */ }
fa_open ("SD:/photos/cat.png");                      // a folder, an app, a document: opened
```

The volume, the wallpaper, the dock — a settings program changes the file and tells the system:

```
#include "systemkit/systemkit.h"

volume_save (7, 0);                                  // 0..10, not muted: kept for the next
start

Wallpaper wp;
wp_load (wp);                                        // SD:/etc/wallpaper.ini
/* change wp ... */
wp_save (wp);

dock_reload ();                                      // the dock reads its settings again
```

Starting a program at every boot — a line of `SD:/etc/autostart` looked for, or added where it belongs (after a given line, else before `preload /boot`, which stays last; Setup's held-back `#setup:` lines respected), and the user told so:

```
#include "systemkit/systemkit.h"
```

```
int r = autostart_ensure ("run stickies", "run agenda", "# Stickies: the pinned notes on the desktop");
/* 2 added, 1 already there (autostart_has), 0 not written */
```

The system's language (the Control Panel's Language & Region): an app's words go through UIKit's `TR ("...")` after `uk_lang_init ()` (section 3); a program with words of its own asks SystemKit:

```
const char *code = locale_language ();                // "en", "fr" (system.ini's language=; none: "en")
for (int i = 0; i < locale_language_count (); i++)    // the languages Onyx speaks, each in its own words
    list->add (locale_language_name (i));
locale_set_language ("fr");                           // kept: the programs started next speak it
locale_set_zone (0);                                  // a time zone (locale_zone_count / _city / _offset): now, and kept
```

The time zones at an instant (the Clock's World tab): `locale_zone_offset (z)` judges the summer time by today's date only; `locale_zone_offset_at (z, utc_minutes)` gives a zone's offset at an exact instant, the hour of the change counted (the EU's at 01:00 UTC, the US' at 02:00 local) — a city's time is its UTC plus that offset, right on the night of a change whatever the local day says:

```
struct kapi_clock_info ci;
if (kapi_clock_info (&ci) == 0 && (ci.flags & KAPI_CLOCK_REALTIME_VALID))
{
    long long utc = ci.utc_us / 60000000;                       // minutes since 1970, UTC
    int ny = 16;                                                // locale_zone_city (16): "New York"
    long long there = utc + locale_zone_offset_at (ny, utc);    // its wall minute now
}
```

The kernel reads `timezone=` once at boot; `locale_zone_sync ()` puts the system's clock on the zone chosen in Language & Region or Setup (`zone=` only — a card with a `timezone=` and no `zone=` is left as it is) when the summer time begins or ends: called once a minute (clockd, the Clock's service, does), on 25 October 2026 at 03:00 CEST the clock becomes 02:00 CET, `timezone=` rewritten (1 changed, 0 not).

The interface's mode and its session (`session.h`, 2026-10-08): the mode — `SESSION_DESKTOP`, `SESSION_POCKET`, `SESSION_CONSOLE` — is `system.ini`'s `shell=`; each mode's programs are its session file `SD:/etc/session/<mode>`. A settings page switches the mode as the Control Panel's Mode applet does: `/bin/session` is started, closes the open programs (each one asks about its unsaved work), and the caller polls it:

```
void *h = session_switch_start (SESSION_POCKET, 0, 0);     // the caller and its parents are kept open, then ended
/* ... each turn: */ if (h && kapi_proc_done (h)) {
    int r = kapi_wait (h); h = 0;
    if (r == SESSION_WAITING_APPS) { char who[512]; session_waiting (who, sizeof who);   // "name\ttitle" lines
        /* ask: Wait -> session_switch_start (m, SESSION_NO_ASK, 0); Force -> ... SESSION_NO_ASK | SESSION_FORCE */ }
}
int m = session_mode ();                                    // the mode chosen (no line: the desktop)
char names[512]; session_programs (m, names, sizeof names); // "menubar\nterminal\n": what its session starts
```

`autostart_has` / `autostart_ensure` look in the autostart and in the session files: a line added after an anchor goes into the file that has it.

The pocket shell (`shell.h`, 2026-10-08): in the pocket mode a program may open one of the pocket shell's screens (`pocketshell` serves the IPC service `shell`); on the desktop nothing serves it and the call answers 0:

```
if (shell_running ()) shell_ask (SHELL_MSG_QUICK);          // quick settings and the notifications (else: our own)
```

The documents opened last (`recent.h`, 2026-10-08): `fa_open` notes every file it opens with its app; an app that opens a file by itself (its own Open dialog) may note it too. The pocket launcher's Recent shows them:

```
recent_doc_add ("SD:/docs/letters-tour.rtf");              // first of SD:/etc/recent-docs (24 at most, a path once)
struct recent_doc d[12]; int n = recent_docs (d, 12);       // the latest first: d[i].path, .date (YYYYMMDD), .time (HHMM)
```

The screen's resolution (`display.h`, 2026-10-09): the sizes offered, the one kept for the next start, and in the console mode each app's own (an emulator's `resolution =`, `SD:/etc/console.ini [screen]`). The Display applet and the console's Display page share it:

```
int w, h;
const char *what = display_mode (8, &w, &h);                // 1920 x 1080, "16:9, Full HD" (English: translate it)
if (kapi_screen_set (w, h) == 0) display_save_size (w, h);  // applied, then kept in SD:/cmdline.txt
display_game_set ("n64emu", 1024, 768);                     // the N64 emulator at 1024 x 768 while it plays
display_game_set ("gcemu", DISPLAY_SYSTEM, 0);              // ... the system's size; DISPLAY_OWN: its app.txt's again
if (display_game_size ("snesemu", &w, &h)) { /* the size consolehome switches to */ }
display_save_option ("opaque", "0");                        // another kernel option of SD:/cmdline.txt (0: removed)
char v[8]; if (display_saved_option ("opaque", v, sizeof v) > 0) { /* "0" */ }
```

The screen at the size of the app in front (`display.h` `display_follow`, 2026-10-26): what the console's home and the pocket shell both do -- an app's own size while it is in front (its `resolution =`, `console.ini`, what it asked with `display_game_ask`), the system's again at home. A shell keeps one `struct display_follow`:

```
static struct display_follow f;  display_follow_init (&f, "pocketshell");
display_follow_launch (&f, "cosmozoom");          // opening it: its size now, before it reads the screen's
display_follow_ask (&f, front, w, h);             // SHELL_MSG_SCREEN received
display_follow_tick (&f, front, names, n);        // every 0.1 s: the app in front ("" at home), the programs running
```

The keyboard and the mouse (`input.h`, 2026-10-09): the layouts on the card, the one in use, taking one (kept for every start), whether it is AZERTY or QWERTZ (a virtual keyboard follows it), the wheel's speed:

```
char maps[INPUT_KEYMAPS_MAX][12]; int n = input_keymaps (maps, INPUT_KEYMAPS_MAX);   // "BE", "DE", "FR", "US"...
input_keymap_set ("BE");                                    // INPUT_KEPT: loaded now and in SD:/etc/autostart
if (input_keymap_kind ("FR") == 1) { /* AZERTY */ }
```

```
input_wheel_save (3);                                   // SD:/etc/theme.txt wheelspeed=
(apply: uk_win_wheel_set)
```

Its part (a header of its own, beside `systemkit.h`)	Subject
`notify.h`	Notifications
`clipboard.h` (`clipproto.h`: its protocol)	The clipboard
`trash.h`	The trash
`fileassoc.h`	Which application opens which file
`volume.h`	The master volume, the output, each program's volume
`wallpaper.h`	The wallpaper's settings and its painter
`dockconf.h`	The dock's settings
`preloadini.h`	The programs loaded ahead at boot
`autostart.h`	The programs started at boot (`SD:/etc/autostart` and the session files)
`applet_proto.h`	A settings applet shown inside the Control Panel
`locale.h`	The system's language and time zone (`SD:/etc/system.ini`)
`session.h`	The interface's mode (desktop, pocket, console) and its session: `SD:/etc/session/<mode>`, the switch
`shell.h`	The pocket shell's screens asked (`shell_ask (SHELL_MSG_HOME / _SWITCHER / _QUICK / _SEARCH / _CLIPBOARD)`, `shell_running ()`): the menu bar's way in pocket
`recent.h`	The documents opened last (`recent_doc_add`, `recent_docs`: `SD:/etc/recent-docs`; `fa_open` notes them)
`display.h`	The screen's resolution: the sizes, the one kept (`SD:/cmdline.txt`), the console's games' own (`SD:/etc/console.ini [screen]`)
`input.h`	The keyboard's layouts (`SD:/etc/keymaps`, kept in `SD:/etc/autostart`) and the mouse wheel's speed

Index

Everything the headers declare, in their order — the details are in each header's part below.

Name	What it does	Header
`notify_action`	A bubble with a title and a text, sent to notifyd (started if it does not run, waited for up to 2 s)	`notify.h`
`notify`	a bubble with a title and a text, no action (notify_action) -> 1 sent, 0 not	`notify.h`
`clip_put`	One copy, n representations (a format and its bytes each).	`clipboard.h`
`clip_get`	The item under the cursor in the first of fmt[] it has (else the newest item that has one of them)	`clipboard.h`
`clip_set_text_n`	copy the n bytes of s as text (a "text" item to clipd, and the kernel's clipboard)	`clipboard.h`
`clip_set_text`	copy the NUL-terminated string s as text (clip_set_text_n)	`clipboard.h`
`clip_get_text`	The clipboard's text into buf (NUL-terminated, truncated to cap-1) -> its length	`clipboard.h`
`clip_set_files`	File / folder paths ('\n'-separated)	`clipboard.h`
`clip_get_file`	The first path into buf	`clipboard.h`
`clip_clear`	After a cut + paste moved the files	`clipboard.h`
`clip_set_image`	w x h pixels 0x00RRGGBB (a row after the other)	`clipboard.h`
`clip_get_image`	-> new[] pixels (delete[] them) and the size, or 0	`clipboard.h`
`ClipItemMsg`	what the widget is told of an item (one message)	`clipproto.h`
`clipc_put32`	v stored at p as 4 bytes, the low one first (little endian)	`clipproto.h`
`clipc_get32`	the 4 bytes at p read as a little-endian number	`clipproto.h`

Name	What it does	Header
`clipc_len`	the length of the string s (0 for a null pointer)	`clipproto.h`
`clipc_eq`	1 if the strings a and b are the same (the case counts), else 0	`clipproto.h`
`clipc_size`	the size of a container of n representations	`clipproto.h`
`clipc_write`	writes it into out (clipc_size bytes)	`clipproto.h`
`clipc_count`	representation i of a container (0..count-1)	`clipproto.h`
`clipc_rep`	representation i of a container (0..count-1)	`clipproto.h`
`trash_ensure`	make the trash's folders (TRASH_DIR, TRASH_FILES, TRASH_INFO) if they are not there	`trash.h`
`trash_move`	Move `path` to the trash.	`trash.h`
`trash_origin`	Original path of trashed item `name` (0 if unknown).	`trash.h`
`trash_restore`	Restore trashed item `name` to its original folder (recreated if needed).	`trash.h`
`trash_purge`	Delete trashed item `name` for good.	`trash.h`
`trash_count`	Number of items in the trash.	`trash.h`
`trash_empty`	Empty the trash (everything in files/ and info/).	`trash.h`
`fa_ext`	Extension of path (after the last '.' of its basename), or "" if none.	`fileassoc.h`
`fa_app_for`	Extension of path (after the last '.' of its basename), or "" if none.	`fileassoc.h`
`fa_is_program`	Extension of path (after the last '.' of its basename), or "" if none.	`fileassoc.h`
`fa_open`	Extension of path (after the last '.' of its	`fileassoc.h`

Name	What it does	Header
	basename), or "" if none.	
`volume_output_word`	An output's word in sound.ini (KAPI_SND_OUT_*).	`volume.h`
`volume_save`	An output's word in sound.ini (KAPI_SND_OUT_*).	`volume.h`
`volume_set_output`	The output chosen (KAPI_SND_OUT_*)	`volume.h`
`volume_input_word`	(v99) The input, a microphone	`volume.h`
`volume_set_input`	The input chosen (KAPI_SND_IN_*, -1	`volume.h`
`mixer_set`	The channel's volume and mute set now (-1	`volume.h`
`volume_restore`	the saved volume -> the kernel (no file	`volume.h`
`Wallpaper`	(a type)	`wallpaper.h`
`wp_defaults`	the wallpaper without a file: voronoi, 28 points, the blues 0x4878B0 and 0x1C2C48, the pattern waves.png	`wallpaper.h`
`wp_eq`	true if a and b are the same string, the case of A..Z ignored	`wallpaper.h`
`wp_colour`	a colour in hexadecimal ("0xRRGGBB", "#RRGGBB", "RRGGBB") -> 0x00RRGGBB; def if v has no hex digit	`wallpaper.h`
`wp_lines`	"key = value" lines of a file	`wallpaper.h`
`wp_key`	"key = value" lines of a file	`wallpaper.h`
`wp_load`	"key = value" lines of a file	`wallpaper.h`
`wp_put`	"key = value" lines of a file	`wallpaper.h`
`wp_put_colour`	"key = value" lines of a file	`wallpaper.h`
`wp_save`	"key = value" lines of a	`wallpaper.h`

Name	What it does	Header
	file	
`wp_isqrt`	the integer square root of n (rounded down)	`wallpaper.h`
`wp_mix`	The colour between a and b (0x00RRGGBB), each channel apart -> a for t 0, b for t 255.	`wallpaper.h`
`wp_paint`	Paint w x h pixels (stride	`wallpaper.h`
`wp_lum`	the luminance 0..255 of a colour: (77 R + 150 G + 29 B) / 256	`wallpaper.h`
`wp_grey_cover`	The grey of a picture (0xAARRGGBB, iw x ih	`wallpaper.h`
`wp_grey_tile`	...	`wallpaper.h`
`wp_multiply`	The colours multiplied by the grey (w x h)	`wallpaper.h`
`DockConf`	(a type)	`dockconf.h`
`dc_copy`	s (0: "") copied into d, cut at cap - 1 characters, NUL-terminated	`dockconf.h`
`dockconf_defaults`	the dock without a file: 6 drawers (Productivity .. Demos), the terminal and fileviewer launchers, desks "1" .. "4"	`dockconf.h`
`dc_split`	A value's two parts	`dockconf.h`
`dockconf_load`	-> false	`dockconf.h`
`dc_put`	-> false	`dockconf.h`
`dockconf_save`	-> false	`dockconf.h`
`dock_running`	The dock takes its settings (and the theme) again	`dockconf.h`
`dock_reload`	The dock takes its settings (and the theme) again	`dockconf.h`
`DockCat`	(a type)	`dockconf.h`
`DockLayout`	(a type)	`dockconf.h`
`dock_category_docked`	The categories that may have a drawer	`dockconf.h`
`dock_layout_load`	dock.ini (or the defaults) merged with the card's	`dockconf.h`

Name	What it does	Header
	apps	
`dock_layout_save`	dock.ini written	`dockconf.h`
`PreloadList`	(a type)	`preloadini.h`
`preload_ini_load`	-> how many programs the file lists (0	`preloadini.h`
`preload_ini_save`	-> 1 written, 0 not	`preloadini.h`
`autostart_has`	Is `cmd` ("run stickies", say) started at boot? A line whose words begin with cmd's words -- blanks before it, more words after it allowed ("run stickies --x")	`autostart.h`
`autostart_ensure`	Make sure `cmd` is started at boot.	`autostart.h`
`locale_ini_get`	system.ini's "key=value"	`locale.h`
`locale_ini_set`	system.ini's "key=value"	`locale.h`
`locale_language_count`		`locale.h`
`locale_language_code`	"en", "fr" ("" out of range)	`locale.h`
`locale_language_name`	in the language itself, UTF-8: "English", "Français"	`locale.h`
`locale_language`	The system's language	`locale.h`
`locale_language_index`	its place in the list	`locale.h`
`locale_set_language`	kept in system.ini -> 1 written (the programs started next take it)	`locale.h`
`locale_zone_count`		`locale.h`
`locale_zone_city`	"Brussels" ("" out of range)	`locale.h`
`locale_zone_summer`	1: the zone is on summer time today (by the clock's date)	`locale.h`
`locale_zone_offset`	minutes from UTC today (the summer time counted)	`locale.h`
`locale_zone_utc`	"UTC+2", "UTC-3:30", "UTC"	`locale.h`
`locale_zone`	the one chosen: system.ini's zone=, else the first of its timezone=	`locale.h`

Name	What it does	Header
	(-1 none)	
`locale_set_zone`	the clock's offset at once, zone= and timezone= kept -> 1 written	`locale.h`
`locale_zone_offset_at`	The zone's offset from UTC at that instant (minutes since 1970, UTC), the hour of the change counted	`locale.h`
`locale_zone_sync`	The zone named by system.ini's zone= (that one only	`locale.h`
`session_modes`	how many modes: 3	`session.h`
`session_mode_name`	"desktop", "pocket", "console" ("" out of range)	`session.h`
`session_mode_find`	the mode of that name (any case) -> SESSION_*, -1 none	`session.h`
`session_mode`	The mode chosen	`session.h`
`session_set_mode`	"shell = " written -> 1 (0: not written)	`session.h`
`session_file`	the mode's session file's path ("SD:/etc/session/pocket") -> its length, 0	`session.h`
`session_programs`	The programs the mode's file starts ("menubar", "dock", "notifyd"...	`session.h`
`session_switch_start`	The switch to mode m (/bin/session switch, started with the flags	`session.h`
`session_switch`	... and waited for -> SESSION_*	`session.h`
`session_waiting`	After SESSION_WAITING_APPS	`session.h`
`session_migrate`	The autostart of a card from before the sessions split, once	`session.h`

systemkit/notify.h

notify.h -- desktop notifications. notify (title, text) sends an IPC message to the "notify" service (the notifyd app), which shows a bubble under the menu bar that fades in, stays ~4 s, then fades out. notifyd is launched on demand if it is not running.

```
  #include "notify.h"
  notify ("File Viewer", "3 items pasted");
  notify_action ("Updates", "3 updates available", "control pkgman");   // a click: that app,
with its
                                                                        // arguments (a
longer stay)

#define NOTIFY_SERVICE  "notify"
#define NOTIFY_MSG_SHOW 1       // payload: title '\0' text '\0' [action '\0': "app args",
run on a click]
#define NOTIFY_MAX  500     // payload bytes (IPC limit 512)
```

A bubble with a title and a text, sent to notifyd (started if it does not run, waited for up to 2 s); action (0 or "" for none) is the "app args" a click on it runs -> 1 sent, 0 not (no notifyd, its mailbox full).

```
int notify_action (const char *title, const char *text, const char *action);
int notify (const char *title, const char *text);   // a bubble with a title and a text, no
action (notify_action) -> 1 sent, 0 not
```

systemkit/clipboard.h

clipboard.h -- the clipboard, shared by every app (docs/clipboard/README.md): the service `clipd` keeps the last CLIP_RING copies (clipproto.h), its widget (the dock's clipboard button) shows them and moves the cursor -- the item Ctrl+V pastes. Header-only, freestanding (the kapi, new[]).

```
  clip_set_text ("hello");            char b[256]; clip_get_text (b, sizeof b);
  clip_set_files ("SD:/a.txt", cut);  int cut; clip_get_file (b, sizeof b, &cut);
  clip_set_image (px, w, h);          int w, h; unsigned *px = clip_get_image (&w, &h);
delete[] px;
  clip_put (fmts, datas, lens, n);    several formats of one copy (Letters: "rtf" and "text")
  clip_get (fmts, nf, got, cap, &data, &len)   the first format of fmts the item has (data:
new[])
```

A copy is also kept by the kernel's clipboard (one text or one path: v40), which is what is pasted when clipd cannot be reached (an older card, the PC's desktop simulator).

the service

clipd's pid; launched when it is not running (0: not there)

copying

One copy, n representations (a format and its bytes each). False: clipd could not be reached.

```
bool clip_put (const char *const *fmt, const void *const *data, const unsigned *len, int n);
```

pasting

The item under the cursor in the first of fmt[] it has (else the newest item that has one of them): its format into got (cap), its bytes into *data (new[]: delete[] it), *len. False: nothing suits, or clipd could not be reached.

```
bool clip_get (const char *const *fmt, int nf, char *got, int cap, unsigned char **data,
unsigned *len);
```

text and paths (the calls every app had)

```
void clip_set_text_n (const char *s, int n);    // copy the n bytes of s as text (a "text"
item to clipd, and the kernel's clipboard)
void clip_set_text (const char *s); // copy the NUL-terminated string s as text
(clip_set_text_n)
```

The clipboard's text into buf (NUL-terminated, truncated to cap-1) -> its length; 0: no text.

```
int clip_get_text (char *buf, int cap);
```

File / folder paths ('\n'-separated): copied, or cut when `cut` != 0 (the paste moves them).

```
void clip_set_files (const char *paths, int cut);
```

The first path into buf; *cut = 1 if it was cut. 0 if no path.

```
int clip_get_file (char *buf, int cap, int *cut);
```

After a cut + paste moved the files: that item goes (clipd), and the kernel's copy.

```
void clip_clear (void);
```

images

w x h pixels 0x00RRGGBB (a row after the other)

```
bool clip_set_image (const unsigned *px, int w, int h);
```

-> new[] pixels (delete[] them) and the size, or 0: no image

```
unsigned *clip_get_image (int *w, int *h);
```

systemkit/clipproto.h

clipproto.h -- the shared clipboard's protocol (docs/clipboard/README.md): what the apps (clipboard.h), the service (apps/clipd) and its widget (apps/clipboard) say to each other.

The service `clipd` (IPC name "clipboard") keeps a ring of CLIP_RING items in its own memory, a cursor on the one Ctrl+V pastes. An item holds one or several representations, each a format (a short name: "text", "rtf", "image", "files", "files-cut", "url", "x-") and its bytes. Messages go through the mailboxes (<= 512 bytes); the bytes through files of RAM: (a copy in the kernel's memory, no mapping kept by anyone): the app writes a container there and says where; for a paste, clipd writes the answer where the app asked and the app waits for that file -- so an app never reads its own mailbox (it may use it for something else).

A container (a file, or the tail of a CLIP_PUT_INLINE): "CLP1", u32 count, then each representation: u16 the format's length, the format, u32 the data's length, the data. An image's data: u32 width, u32 height, then width * height pixels 0x00RRGGBB.

```
#define CLIP_SERVICE    "clipboard"
#define CLIP_RING   10
#define CLIP_DIR    "RAM:/clip"         // the transfers' files

enum
{
```

```
    // an app -> clipd
    CLIP_PUT = 1,           // "path\0source\0": the container in that file (clipd removes
it)
    CLIP_PUT_INLINE = 2,        // "source\0" + a container: a small copy, whole in the
message
    CLIP_GET = 3,           // "reply path\0fmt\0fmt\0...\0": the cursor's item in the first
                    // format given it has -- else the newest item that has one --,
                    // written to the reply path (a container of 0 or 1 representation)
    CLIP_DROP_CUT = 4,      // the cursor's item, if it is a cut (files-cut): deleted
(pasted)
    // the widget -> clipd
    CLIP_LIST = 5,          // -> CLIP_ITEM ... CLIP_END, to the sender
    CLIP_CURSOR = 6,        // u32 id: the cursor there
    CLIP_DELETE = 7,        // u32 id
    CLIP_CLEAR = 8,         // every item
    CLIP_SUBSCRIBE = 9,     // the sender told CLIP_CHANGED at every change
    // clipd -> the widget
    CLIP_ITEM = 100,        // struct ClipItemMsg
    CLIP_END = 101,         // u32 count
    CLIP_CHANGED = 102
};
```

what the widget is told of an item (one message)

```
struct ClipItemMsg
{
    unsigned id, size;      // size: the main representation's bytes
    unsigned w, h;          // an image's
    unsigned char cursor, nfiles, hour, minute;
    char kind[16];          // the main format ("text", "image", "files", "files-cut", "rtf",
"url"...)
    char source[32];        // the app it came from
    char preview[400];      // a line of text (a text's start, the files' names, the URL)
};
```

the container

v stored at p as 4 bytes, the low one first (little endian)

```
void clipc_put32 (unsigned char *p, unsigned v);
```

the 4 bytes at p read as a little-endian number

```
unsigned clipc_get32 (const unsigned char *p);
```

the length of the string s (0 for a null pointer)

```
int clipc_len (const char *s);
```

1 if the strings a and b are the same (the case counts), else 0

```
int clipc_eq (const char *a, const char *b);
```

the size of a container of n representations

```
unsigned clipc_size (const char *const *fmt, const unsigned *len, int n);
```

writes it into out (clipc_size bytes); -> the bytes written

```
unsigned clipc_write (unsigned char *out, const char *const *fmt, const void *const *data,
const unsigned *len, int n);
```

representation i of a container (0..count-1): its format into fmt (cap), *data, *len -> 1 / 0

```
int clipc_count (const unsigned char *c, unsigned n);
int clipc_rep (const unsigned char *c, unsigned n, int i, char *fmt, int cap, const unsigned
char **data, unsigned *len);
```

systemkit/trash.h

trash.h -- the Onyx trash (freedesktop-style layout on the SD card):

```
  SD:/.Trash/files/<name>             the trashed file or folder
  SD:/.Trash/info/<name>.trashinfo    "Path=<original path>"
```

trash_move (path) moves an item there (a clash gets " (2)"-style names); trash_restore (name) puts it back at its original path (a free variant if that path is taken now); trash_purge (name) deletes one item for good; trash_empty () deletes everything. /bin/rm still deletes for real -- the trash is for interactive deletes (File Viewer).

```
#define TRASH_DIR   "SD:/.Trash"
#define TRASH_FILES "SD:/.Trash/files"
#define TRASH_INFO  "SD:/.Trash/info"
void trash_ensure (void);   // make the trash's folders (TRASH_DIR, TRASH_FILES, TRASH_INFO)
if they are not there
```

Move `path` to the trash. Returns true on success.

```
bool trash_move (const char *path);
```

Original path of trashed item `name` (0 if unknown).

```
bool trash_origin (const char *name, char *out, int cap);
```

Restore trashed item `name` to its original folder (recreated if needed). Returns true on success; `where` (optional) receives the restored path.

```
bool trash_restore (const char *name, char *where = 0, int wcap = 0);
```

Delete trashed item `name` for good.

```
bool trash_purge (const char *name);
```

Number of items in the trash.

```
int trash_count (void);
```

Empty the trash (everything in files/ and info/).

```
int trash_empty (void);
```

systemkit/fileassoc.h

fileassoc.h -- file associations (SD:/etc/fileassoc.ini: "ext = app" lines).

```
  fa_app_for (path, app, cap)  the app associated with path's extension (0 = none)
  fa_open (path)               open it: a folder in the File Viewer, a .app bundle
                               or an ELF program runs, a file in its associated app
                               (SD:apps/<app>.app/main <path>) -- a NEW instance.
```

```
#define FA_INI      "SD:/etc/fileassoc.ini"
```

Extension of path (after the last '.' of its basename), or "" if none.

```
const char *fa_ext (const char *path);
bool fa_app_for (const char *path, char *app, int cap);
bool fa_is_program (const char *path);
bool fa_open (const char *path);
```

systemkit/volume.h

volume.h -- the master volume (kapi v60 sound_volume: 0..10 and mute), kept in SD:/etc/sound.ini ("volume = 7", "mute = 0") so it comes back after a reboot: the menu bar applies it at start. Used by the menu bar's volume box and /bin/volume. (v84) The file also says which output plays ("output = auto | jack | usb | hdmi": the kernel reads it when the sound first starts, kapi sound_output changes it at once): volume_save keeps that line.

```
#define VOLUME_INI  "SD:/etc/sound.ini"
```

An output's word in sound.ini (KAPI_SND_OUT_*).

```
const char *volume_output_word (int out);
void volume_save (int vol, int mute);
```

The output chosen (KAPI_SND_OUT_*): applied now, kept in the file -> sound_output's result.

```
int volume_set_output (int out);
```

(v99) The input, a microphone: "input = auto | none | usb | i2s" and "input_gain = " in the same file.

```
const char *volume_input_word (int in);
```

The input chosen (KAPI_SND_IN_*, -1: kept) and its gain (dB 0..48, -1: kept): the kernel's from the next recording, kept in the file -> sound_input's SELECT result (-KAPI_ENOSYS: a kernel before v99).

```
long volume_set_input (int in, int gain_db);
```

the mixer (kapi v85): a program's own volume

Every program that plays has a channel (kapi_sound_clients); its volume (0..100) and its mute are remembered by its name in SD:/etc/mixer.ini ("media = 60", "media.mute = 1"), which the kernel reads when the sound first starts.

```
#define MIXER_INI   "SD:/etc/mixer.ini"
```

The channel's volume and mute set now (-1: kept) and written to the file -> volume | 0x100 muted, -1.

```
int mixer_set (const struct kapi_sound_client *c, int volume, int mute);
```

the saved volume -> the kernel (no file: full, not muted)

```
void volume_restore (void);
```

systemkit/wallpaper.h

wallpaper.h -- the desktop's wallpaper settings (SD:/etc/wallpaper.ini) and its painter, shared by apps/voronoy (it paints the wallpaper at boot, and again when the Theme applet applies one) and the Theme applet (its preview, and the file it writes). Integer only (the apps' default).

```
    mode      = voronoi      voronoi (cells), gradient (two colours), bubbles (a gradient
with
                             soft bubbles over it), solid (one colour), image (a picture
file),
                             pattern (a grey picture coloured by the gradient: multiplied)
    color     = 0x4878B0     voronoi's base colour, the solid colour, the gradient's first
    color2    = 0x1C2C48     the gradient's second (gradient, bubbles, pattern)
    direction = vertical     the gradient: vertical (top to bottom) or horizontal (left to
right)
    points    = 28           voronoi's cells (1..64)
    image     = SD:/x.jpg    image: the picture (BMP GIF PNG JPEG PCX WebP), painted by
    style     = cover        apps/imageview --background: cover (the screen filled, centred)
                             or tile (repeated from the top left)
    tint      = no           image: yes -- its grey multiplies `color` (a tinted picture:
                             painted by voronoy itself), no -- shown as it is
    pattern   = SD:/wallpapers/waves.png   pattern: the grey picture (tools/gen_wallpapers.py
                             makes the shipped ones, SD:/wallpapers): each pixel's grey
                             multiplies the gradient -- white is the colour itself -- as
                             "cover" fills the screen (wp_grey_cover, wp_multiply)
```

No file: SD:/apps/voronoy.app/config.ini's base / points (before the Theme applet had it).

```
#define WALLPAPER_INI   "SD:/etc/wallpaper.ini"

#define WALLPAPER_DIR   "SD:/wallpapers"        // the patterns

enum { WP_VORONOI, WP_GRADIENT, WP_BUBBLES, WP_SOLID, WP_IMAGE, WP_PATTERN, WP_NMODES };

struct Wallpaper
{
    int mode;
    unsigned c1, c2;
    int vertical;           // the gradient's direction (1: top to bottom)
    int points;
    char image[200];
    int tile;           // image: 1 tiled, 0 cover
    int tint;           // image: 1 its grey times c1, 0 as it is
    char pattern[200];      // pattern: the grey picture
};
void wp_defaults (Wallpaper &w);    // the wallpaper without a file: voronoi, 28 points, the
blues 0x4878B0 and 0x1C2C48, the pattern waves.png
bool wp_eq (const char *a, const char *b);  // true if a and b are the same string, the case
of A..Z ignored
unsigned wp_colour (const char *v, unsigned def);   // a colour in hexadecimal ("0xRRGGBB",
"#RRGGBB", "RRGGBB") -> 0x00RRGGBB; def if v has no hex digit
```

"key = value" lines of a file: fn (key, value) for each. -> false: no file.

```
bool wp_lines (const char *path, void (*fn) (const char *k, const char *v, void *ctx), void
*ctx);
void wp_key (const char *k, const char *v, void *ctx);
void wp_load (Wallpaper &w);
int wp_put (char *o, int p, int cap, const char *s);
int wp_put_colour (char *o, int p, int cap, unsigned c);
bool wp_save (const Wallpaper &w);
```

the painter

```
unsigned wp_isqrt (unsigned n); // the integer square root of n (rounded down)
```

The colour between a and b (0x00RRGGBB), each channel apart -> a for t 0, b for t 255.

```
static inline unsigned wp_mix (unsigned a, unsigned b, int t)       // t 0..255: a -> b
{
int r = (int) ((a >> 16) & 255) + ((int) ((b >> 16) & 255) - (int) ((a >> 16) & 255)) * t /
255;
int g = (int) ((a >> 8) & 255) + ((int) ((b >> 8) & 255) - (int) ((a >> 8) & 255)) * t / 255;
int c = (int) (a & 255) + ((int) (b & 255) - (int) (a & 255)) * t / 255;
}
// Voronoi's tint: the base colour darker near a cell's seed, lighter at its edges
(SimpleOS's).
SK_API unsigned wp_tint (unsigned base, unsigned dist);
```

Paint w x h pixels (stride: a row) -- not an image (apps/imageview does those); a pattern: its gradient (then wp_multiply by the pattern's grey). div: the cells computed at 1 / div of the resolution (voronoi: 2 for the screen, 1 for a small preview). yield: called now and then (a long job: kapi_yield), or 0.

```
void wp_paint (unsigned *dst, int w, int h, int stride, const Wallpaper &wp, unsigned seed,
int div, void (*yield) (void));
```

a pattern: a grey picture multiplying the colours

```
unsigned wp_lum (unsigned c);   // the luminance 0..255 of a colour: (77 R + 150 G + 29 B) /
256
```

The grey of a picture (0xAARRGGBB, iw x ih: its luminance) over w x h as "cover" lays it (the area filled, centred, the rest cut off): the average of the pixels under each one where it shrinks (a small preview), bilinear where it grows (a bigger screen).

```
void wp_grey_cover (const unsigned *img, int iw, int ih, unsigned char *out, int w, int h);
```

... tiled from the top left instead (a tinted picture's "tile" style): the picture at the screen's scale (sw x sh: the screen w x h stands for -- a small preview's).

```
void wp_grey_tile (const unsigned *img, int iw, int ih, unsigned char *out, int w, int h, int
sw, int sh);
```

The colours multiplied by the grey (w x h): white keeps them, black makes them black.

```
void wp_multiply (unsigned *dst, int w, int h, int stride, const unsigned char *grey);
```

systemkit/dockconf.h

dockconf.h -- the dock's settings (SD:/etc/dock.ini), shared by the dock (apps/dock) and the Control Panel's Panel applet (apps/dockconf), which writes them and tells the dock to read them again (IPC service "dock", DOCK_MSG_RELOAD). The file, one setting a line:

```
    drawer   = Productivity, tinypad    a drawer: the apps' group (the "category" of their
                                        app.txt) and its main app (its icon is the drawer's;
a
                                        click on it starts the app, the strip above opens the
                                        drawer). In this order, left to right.
    launcher = terminal                 a quick launcher after the drawers (a click starts
it,
                                        or brings it back)
    desk     = Main                     a workspace (virtual desktop) and its name: 1 to
                                        DOCK_MAXDESKS of them
    hidden   = Demos                    (2026-10-06) a category that has no drawer
```

Since 2026-10-06 the dock has a drawer for EVERY category of the card's apps (their app.txt), but Shell, Settings and Emulators (dock_category_docked), and those hidden: dock_layout_load reads the file's drawers as the categories' order and main apps, and adds the categories it does not name at the end. DockConf / dockconf_load / dockconf_save (8 drawers, what the file says) stay for the programs built before (the table is append-only); the dock and the Panel applet use DockLayout.

```
#define DOCK_INI         "SD:/etc/dock.ini"
#define DOCK_SERVICE         "dock"
#define DOCK_MSG_RELOAD      1       // (no payload) read dock.ini and theme.txt again
#define DOCK_MAXDRAWERS      8
#define DOCK_MAXLAUNCHERS    6
#define DOCK_MAXDESKS        6

struct DockDrawer { char cat[24]; char app[32]; };
struct DockConf
{
    DockDrawer drawer[DOCK_MAXDRAWERS]; int ndrawers;
    char launcher[DOCK_MAXLAUNCHERS][32]; int nlaunchers;
    char desk[DOCK_MAXDESKS][24]; int ndesks;
};
void dc_copy (char *d, const char *s, int cap); // s (0: "") copied into d, cut at cap - 1
characters, NUL-terminated
void dockconf_defaults (DockConf &c);   // the dock without a file: 6 drawers
(Productivity .. Demos), the terminal and fileviewer launchers, desks "1" .. "4"
```

A value's two parts: "Productivity, tinypad" -> "Productivity" and "tinypad".

```
void dc_split (const char *v, char *a, int acap, char *b, int bcap);
```

-> false: no file (the defaults)

```
bool dockconf_load (DockConf &c);
int dc_put (char *o, int p, int cap, const char *s);
bool dockconf_save (const DockConf &c);
```

The dock takes its settings (and the theme) again: it is started again -- a dock that only read dock.ini again (DOCK_MSG_RELOAD) showed a new drawer or launcher but did not always act on it (a click started nothing, a new group was not seen); a new one has them all. Nothing when no dock runs (the user may have taken it out of etc/autostart).

```
int dock_running (void);
void dock_reload (void);
```

the dock's layout: every category, its order, its main app, hidden or not (2026-10-06)

```
#define DOCK_MAXCATS        32
struct DockCat
{
    char cat[24];               // the category (the app.txt's)
    char app[32];               // its main app (the drawer's icon; "": the first of its apps
by name)
    int  hidden;                // 1: no drawer for it
    int  napps;             // how many apps of the card are in it (0: named by dock.ini
only)
    int  reserved[2];
};
struct DockLayout
{
    DockCat cat[DOCK_MAXCATS]; int ncats;   // in the dock's order, left to right
```

```
    char launcher[DOCK_MAXLAUNCHERS][32]; int nlaunchers;
    char desk[DOCK_MAXDESKS][24]; int ndesks;
    int reserved[8];
};
```

The categories that may have a drawer: all but Shell, Settings and Emulators (the desktop's parts, the Control Panel's applets, the Game Library's emulators).

```
bool dock_category_docked (const char *cat);
```

dock.ini (or the defaults) merged with the card's apps: every category they have, those the file does not name added at the end (an app without a category: "Other"); a main app that is not there any more: "". -> false: no file (the default order: Productivity, Internet, Graphics, Programming, Games, Multimedia, Demos, System, then the others by name).

```
bool dock_layout_load (DockLayout &l);
```

dock.ini written: every category in order (the hidden ones too, and "hidden = " lines for them), the launchers, the workspaces.

```
bool dock_layout_save (const DockLayout &l);
```

systemkit/preloadini.h

preloadini.h -- SD:/etc/preload.ini: the programs loaded ahead at boot and kept in memory (`preload /boot`, the last line of /etc/autostart; the Control Panel's Preload applet writes the file). One program a line -- a path, an app's name (apps/.app/main) or a /bin tool's, as preload's arguments (bin/imgname.h); blank lines and lines starting with '#' or ';' are skipped.

MIT License -- Copyright (c) 2026 Stéphane Wegener and the Onyx contributors. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED.

```
#define PRELOAD_INI "SD:/etc/preload.ini"
#define PRELOAD_MAX 32
#define PRELOAD_NAME    128

struct PreloadList
{
    int  n;
    char prog[PRELOAD_MAX][PRELOAD_NAME];
};
```

-> how many programs the file lists (0: none, or no file)

```
int preload_ini_load (struct PreloadList *l);
```

-> 1 written, 0 not

```
int preload_ini_save (const struct PreloadList *l);
```

systemkit/autostart.h

autostart.h -- SD:/etc/autostart, the programs started at boot (one shell command a line: `run agenda`, `keyb FR`, `preload /boot` the last one), as an app that offers "start it at every boot" changes it: a line looked for, a line added where it belongs -- never moved, never removed, every other line kept byte for byte. Setup's held-back lines count: on a card whose first-run wizard has not ended, a line is written "#setup: " and given back by Setup at its end (apps/setup) -- such a line is "there", and a line added after one of them is held back the same way. Used by Notes (View > Show Stickies on the Desktop) and the Clock (clockd). Since the sessions (session.h, 2026-10-08) the programs started at boot are in two places: SD:/etc/autostart, the system's part (the services), and the session files SD:/etc/session/ (desktop, pocket, console: the interface's programs -- the menu bar, the dock, the agenda...). Both functions look in all of them: a line is "there" in any one, and a line is added to the file whose `after` line it follows (Stickies after the agenda: SD:/etc/session/desktop). C and C++.

MIT License -- Copyright (c) 2026 Stéphane Wegener and the Onyx contributors. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED.

```
#define AUTOSTART_PATH  "SD:/etc/autostart"
#define AUTOSTART_SESSIONS  "SD:/etc/session/"  // + "desktop", "pocket", "console": the
sessions' files
#define AUTOSTART_MAX   16384           // a bigger file is left alone (never cut)
```

Is `cmd` ("run stickies", say) started at boot? A line whose words begin with cmd's words -- blanks before it, more words after it allowed ("run stickies --x") -- either active or held back by Setup ("#setup: run stickies"), in the autostart or in a session's file. A plain comment ("# run stickies") is not. -> 1 there, 0 not (or no file)

```
int autostart_has (const char *cmd);
```

Make sure `cmd` is started at boot. Nothing written when autostart_has (cmd). Else the line `cmd`, after the comment line `comment` ("# ...", or 0: none), is INSERTED: right after the first line whose command starts with the words `after` (e.g. "run agenda"; 0: no such rule) -- looked for in the autostart, then in the sessions' files: in the file that has it --, with that line's "#setup: " when it has one (held back as it is); else in the autostart, just before the first `preload` line (it stays the last); else at its end (no file: a new one). -> 1 already there, 2 added, 0 not written (the file too big, the write failed)

```
int autostart_ensure (const char *cmd, const char *after, const char *comment);
```

systemkit/locale.h

locale.h -- the system's language and region, kept in SD:/etc/system.ini:

```
  language = fr        the language of the programs' words ("en" when no line):
uikit/lang.h's TR () reads
                       it (uk_lang_init), a program with words of its own asks
locale_language ()
  zone     = Brussels  the time zone's city (locale_zone_*), beside "timezone=" -- its offset
in minutes,
```

```
                        the summer time counted, which the kernel reads at boot (and
locale_zone_sync keeps
                        right when the summer time begins or ends)
```

Chosen in the Control Panel's Language & Region applet and in Setup (the first-run wizard). A language is taken by a program when it starts.

MIT License -- Copyright (c) 2026 Stéphane Wegener and the Onyx contributors. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED.

```
#define LOCALE_INI  "SD:/etc/system.ini"
```

system.ini's "key=value": the value in out ("" none) -> 1 found; the key's line replaced (else added) -> 1 written.

```
int locale_ini_get (const char *key, char *out, int cap);
int locale_ini_set (const char *key, const char *value);
```

the languages Onyx speaks

```
int locale_language_count (void);
const char *locale_language_code (int i);   // "en", "fr" ("" out of range)
const char *locale_language_name (int i);   // in the language itself, UTF-8: "English",
"Français"
```

The system's language: one of the codes ("en" when none, or an unknown one, is said). Read from the file at each call: a program keeps it.

```
const char *locale_language (void);
int locale_language_index (void);       // its place in the list
int locale_set_language (const char *code); // kept in system.ini -> 1 written (the programs
started next take it)
```

the time zones

```
int locale_zone_count (void);
const char *locale_zone_city (int z);       // "Brussels" ("" out of range)
int locale_zone_summer (int z);         // 1: the zone is on summer time today (by the
clock's date)
int locale_zone_offset (int z);         // minutes from UTC today (the summer time counted)
void locale_zone_utc (int z, char *out, int cap);   // "UTC+2", "UTC-3:30", "UTC"
int locale_zone (void);             // the one chosen: system.ini's zone=, else the first of
its timezone= (-1 none)
int locale_set_zone (int z);            // the clock's offset at once, zone= and timezone=
kept -> 1 written
```

The zone's offset from UTC at that instant (minutes since 1970, UTC), the hour of the change counted: the EU's summer time from the last Sunday of March 01:00 UTC to the last Sunday of October 01:00 UTC; the US' from the second Sunday of March 02:00 local standard time to the first Sunday of November 02:00 local summer time -> minutes (0 for a zone out of range). Judged from UTC, which never goes

back: a city's time is UTC + locale_zone_offset_at (city, UTC), right on the night of a change whatever the local day says.

```
int locale_zone_offset_at (int z, long long utc_minutes);
```

The zone named by system.ini's zone= (that one only: never locale_zone ()'s guess from timezone=), its offset now (kapi_clock_info's UTC, locale_zone_offset_at) given to the clock (kapi_set_timezone) and to system.ini's timezone= when it differs from the clock's (kapi_clock_info's tz_minutes) -> 1 changed, 0 not (no zone= or an unknown city, no real date yet, already right). Called once a minute (clockd does), the clock follows the summer time by itself: on the night it ends, 03:00 becomes 02:00 at 01:00 UTC.

```
int locale_zone_sync (void);
```

systemkit/session.h

session.h -- the interface's mode and its session (docs/POCKETUI-TECH-STUDY.md section 8). Onyx has three modes:

```
  desktop   Elegant, the desktop (the menu bar, the dock, the agenda...)
SD:/etc/session/desktop
  pocket    PocketUI, one app at a time on a small screen
SD:/etc/session/pocket
  console   PocketUI's console mode: the games, a television, a pad
SD:/etc/session/console
```

The mode is SD:/etc/system.ini's "shell =" (no line: desktop), which the kernel reads to start the matching graphics server. The SESSION is the mode's programs: SD:/etc/autostart keeps the system's part (the services: clockd, pkgd, clipd, printd, telnetd...) and one line, `session`, where /bin/session runs the file of the mode (the same syntax as the autostart: `run menubar`, `#setup: run dock`, `sleep 1`...). Switching the mode closes the open programs (an unsaved document asked), ends the session's programs, writes "shell =", has the kernel start the other server (KAPI_WS_SWITCH) and runs the other mode's file: /bin/session does it, asked by the Control Panel's Mode applet (modeconf) or typed (`session switch pocket`). A card from before the sessions keeps the desktop's lines in its autostart until `pkg commit` (at boot) moves them into SD:/etc/session/desktop once (session_migrate). C and C++.

MIT License -- Copyright (c) 2026 Stéphane Wegener and the Onyx contributors. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED.

```
#define SESSION_DESKTOP     0   // the modes (UIKit's UK_MODE_* are the same numbers)
#define SESSION_POCKET      1
#define SESSION_CONSOLE     2
#define SESSION_DIR     "SD:/etc/session"   // SESSION_DIR "/<mode's name>": the mode's
programs
#define SESSION_TOOL        "SD:/bin/session"
#define SESSION_WAITING     "SD:/tmp/session.wait"  // the programs that did not close
(session_waiting)
```

session_switch's flags

```
#define SESSION_FORCE       1   // the programs still open after the wait are ended (their
documents lost)
#define SESSION_NO_ASK      2   // they are not asked again (they were: their question is
up), only waited for
```

session_switch's answers (/bin/session switch's exit status)

```
#define SESSION_SWITCHED    0   // the mode asked for runs, its programs started
#define SESSION_FELL_BACK   1   // its server failed: the desktop runs ("shell =" put back to
desktop)
#define SESSION_WAITING_APPS    2   // programs did not close in time (session_waiting):
nothing switched
#define SESSION_REFUSED     3   // a full-screen program has the display, or a switch is
under way
#define SESSION_BAD     4   // an unknown mode, wrong arguments
#define SESSION_FAILED      5   // system.ini not written, no server took the display, the
tool missing

int session_modes (void);               // how many modes: 3
const char *session_mode_name (int m);      // "desktop", "pocket", "console" ("" out of
range)
int session_mode_find (const char *name);       // the mode of that name (any case) ->
SESSION_*, -1 none
```

The mode chosen: system.ini's "shell =" (no line, or an unknown word: SESSION_DESKTOP). Read at each call. (The server running may differ: its server failed at boot and the kernel started Elegant -- UIKit's uk_win_server says which runs.)

```
int session_mode (void);
int session_set_mode (int m);           // "shell = <name>" written -> 1 (0: not written)
int session_file (int m, char *out, int cap);   // the mode's session file's path
("SD:/etc/session/pocket") -> its length, 0
```

The programs the mode's file starts ("menubar", "dock", "notifyd"...: the names their processes have; a `run <app>` line gives the app, another line its /bin tool; Setup's held-back "#setup: " lines count), one a line into out -> how many (0: no file).

```
int session_programs (int m, char *out, int cap);
```

The switch to mode m (/bin/session switch, started with the flags; keep_pid: a program kept open meanwhile -- a Control Panel applet's host --, 0 none; the caller and its parents always are, then ended once the new session runs) -> its process handle (kapi_proc_done, kapi_wait: SESSION_* answer), 0 not started.

```
void *session_switch_start (int m, int flags, int keep_pid);
int session_switch (int m, int flags);      // ... and waited for -> SESSION_*
```

After SESSION_WAITING_APPS: the programs that did not close, "\t" a line -> how many.

```
int session_waiting (char *out, int cap);
```

The autostart of a card from before the sessions split, once: the desktop's lines (voronoy, Setup and its "#setup#" lines, menubar, notifyd, dock, agenda, stickies, wifimenu, imageview -- with the comment lines just above each) moved into SD:/etc/session/desktop (made anew from them), a line `session` where the first of them was, the old file kept as SD:/etc/autostart.old; every other line where it was. Nothing done when the autostart has a `session` line already. -> 1 split, 0 nothing to do, -1 not written (the card as it was).

```
int session_migrate (void);
```

systemkit/applet_proto.h

applet_proto.h -- the Control Panel's applets (apps/control): an applet is a uikit app shown INSIDE the Control Panel's window instead of in a window of its own. The host (apps/control) makes a shared surface (kapi v35) the size of its pane and starts the applet with the arguments "--applet "; the applet's uikit::Root then draws into that surface (uikit/root.cpp, the applet mode); the host copies it into its window when told and sends it the pointer and the keys. Messages over the kernel's mailboxes (kapi v35 / v40, <= 512 bytes), from / to the pids:

```
  applet -> host   AP_HELLO    int w, h: the applet is up (its Root the surface's size)
                   AP_PRESENT  int x, y, w, h: it drew (0 0 0 0: all of it) -- copy it
                   AP_EXIT     it is ending: stop reading the surface
                   AP_THEME    it applied a new theme (SD:/etc/theme.txt): the host takes it
                               and starts the applet again (in the new colours)
  host -> applet   AP_PTR      struct ApPtr: the pointer (a GUI_EVENT_PTR_* event, pane
                               coordinates; x < 0: it left the pane)
                   AP_KEY      struct ApKey: a key typed (a character or a KEY_* code)
                   AP_CLOSE    please end (the user went back to the applets' list)
  anyone -> host   AP_OPEN     the target's name (an applet's app, NUL-terminated): show that
applet -- a second
                               `control <target>` asks the one running (2026-10-08: pocket's
Settings)
```

The host is the IPC service AP_SERVICE: an applet whose host is gone ends by itself. Another host (Mail, showing Web as its HTML view) adds its own service's name: "--applet "; its own message types go to the applet's uk_applet_on_message (uikit/root.h; Jet's web view: Apps/jet/webview_proto.h, types 60..79). The surface's frames live as long as either process maps it (kernel v65: its users).

```
#define AP_SERVICE  "control"

enum
{
    AP_HELLO = 40, AP_PRESENT = 41, AP_EXIT = 42, AP_THEME = 43, AP_OPEN = 44,
    AP_PTR = 50, AP_KEY = 51, AP_CLOSE = 52
};

struct ApPtr { int event, x, y, buttons, changed, wheel; };
struct ApKey { int key; unsigned mods; };
```